

ECE 315

Steven Knudsen, PhD, PEng

Lecture 26 :

- Pico VGA Example
- DMA + PIO Example



UNIVERSITY
OF ALBERTA

Last Lecture

- DMA Example (RP2040)
- Programmable I/O (PIO)

This Lecture

- Pico VGA Example
- DMA + PIO Example

Learning Objectives

- Appreciate two examples of using DMA and PIO
- Understand VGA Signaling and Timing
- Apply State Machine Concepts to VGA Generation
- Describe the RP2040 VGA Implementation Architecture
- Understand how to use PIO for PWM Waveform Generation
- Implement DMA + PIO Interaction

Video Graphics Array (VGA)



- Refers to both a *de facto* standard and hardware computer display
 - These days always uses 15-pin D-subminiature connector and 640x480 pixel resolution
- Very quickly and widely adopted, still in use
- Various extensions to higher resolutions, e.g., up to 2048x1536
- Comprises 3 main binary TTL (5V) signals
 - Horizontal sync
 - Vertical sync
 - RGB levels



Wikipedia contributors. "Video Graphics Array." Wikipedia, The Free Encyclopedia. Wikipedia, The Free Encyclopedia, 14 Nov. 2025. Web. 7 Dec. 2025.

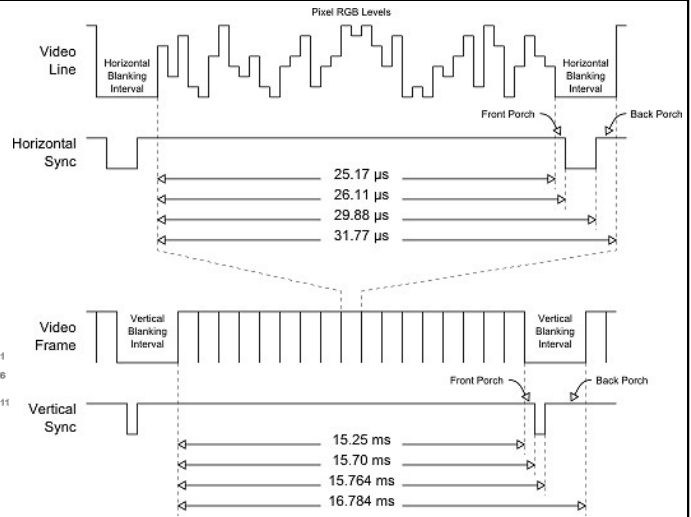
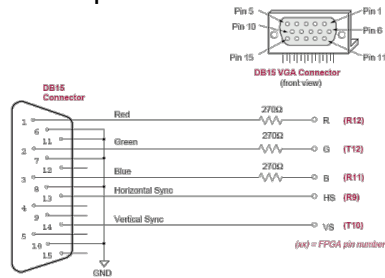
<https://vanhunteradams.com/Pico/VGA/VGA.html> 5

DB15 connector - By Evan-Amos - Own work, Public Domain,
<https://commons.wikimedia.org/w/index.php?curid=11339219>

VGA signals

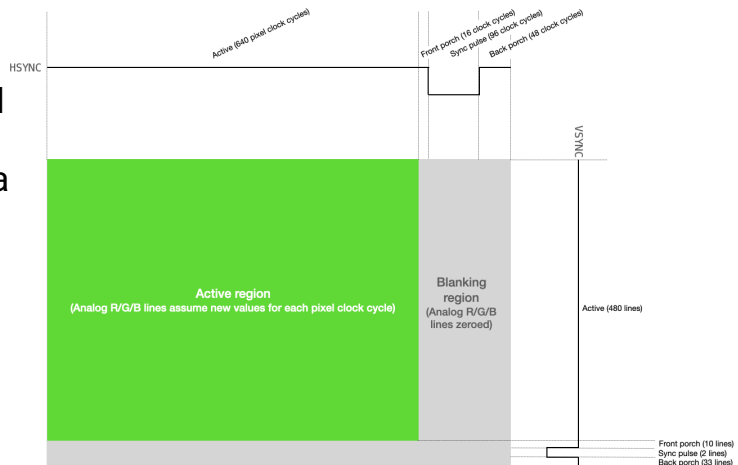
General idea;

- Within every vertical sync interval, 480 horizontal lines are drawn
- On each line, 640 horizontal RGB levels are output



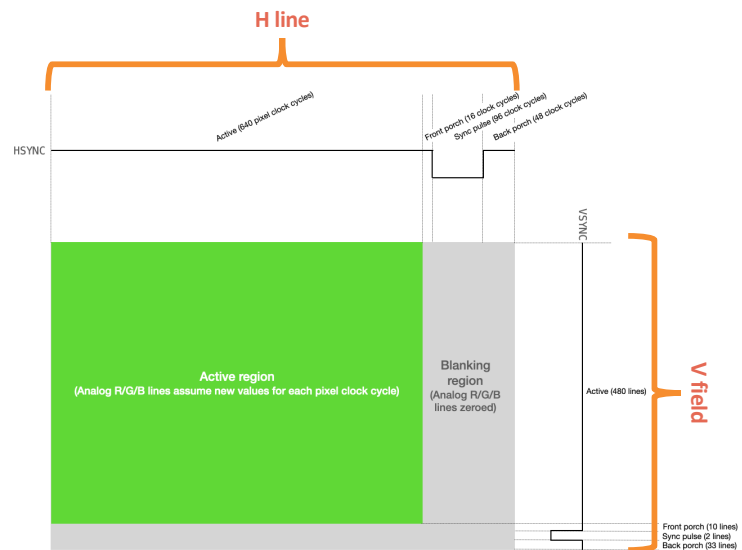
VGA signals

- Slightly more complicated since a few extra horizontal pixels and extra vertical lines are needed
 - So-called front and back porches
- Historical due to Cathode Ray Tube (CRT) technology



VGA signals

- A line (**H line**) is a row of pixels
- A field (**V field**) is a set of lines



Number of H and V lines

"VGA industry standard" 640x480 pixel mode

General characteristics

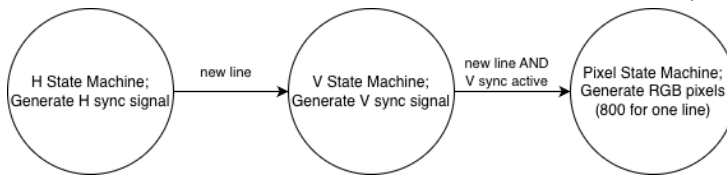
- Clock frequency 25.175 MHz
- Line frequency 31469 Hz
- Field frequency 59.94 Hz

One Line	One Field
8 pixels front porch	2 lines front porch
96 pixels horizontal sync	2 lines vertical sync
40 pixels back porch	25 lines back porch
8 pixels left border	8 lines top border
640 pixels video	480 lines video
8 pixels right border	8 lines bottom border
800 pixels total per line	525 lines total per field

State Machine for VGA

- Very simple state machine
- H and V state machines count and toggle the sync signals accordingly
- Pixel state machine outputs 800 RGB values when v sync is active

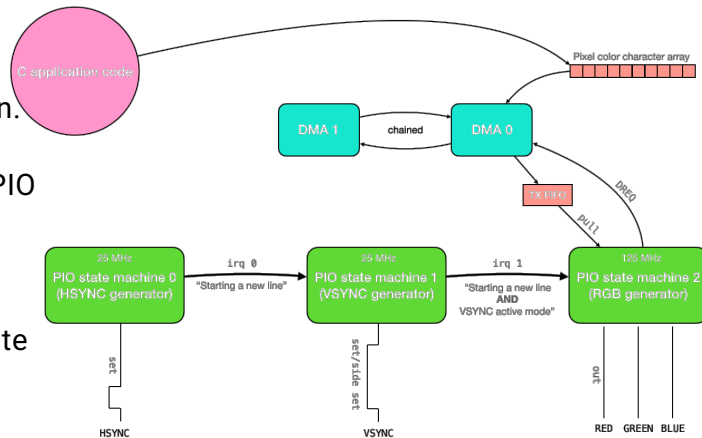
One Line	One Field
8 pixels front porch	2 lines front porch
96 pixels horizontal sync	2 lines vertical sync
40 pixels back porch	25 lines back porch
8 pixels left border	8 lines top border
640 pixels video	480 lines video
8 pixels right border	8 lines bottom border
800 pixels total per line	525 lines total per field



<https://vanhunteradams.com/Pico/VGA/VGA.html> 10

RP2040 (Pico) Implementation

- Three PIO state machines (synchronized with one another via interrupts) drive a VGA screen.
- The pixel data to draw on the screen is communicated to the PIO state machines via DMA
- Pixel data is stored in a global character array
- Application code need only update the array



H sync generation

- PIO assembly
- Clock is set to 25.172 MHz
- The scratch registers are 32-bit, more than big enough to count 640+16, but can't be directly set
 - Can copy from OSR to x
- `activeporch` loop is 656 cycles
- Can see how the H sync signal is generated by this
- Interrupt signal generated at the end before it wraps

```
pull block ; Pull from FIFO to OSR (only happens once)
.wrap_target ; Program wraps to here

; ACTIVE + FRONTPORCH
mov x, osr ; Copy value from OSR to x scratch register
activeporch:
    jmp x-- activeporch ; Remain high in active mode and front
porch

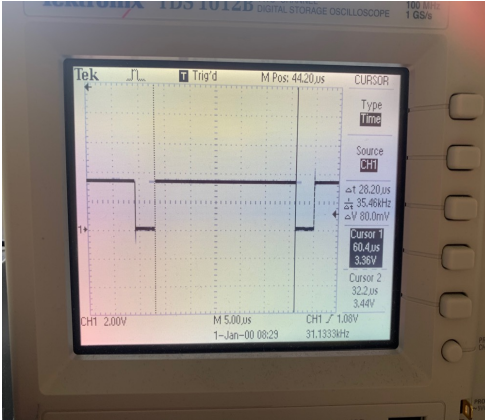
; SYNC PULSE
pulse:
    set pins, 0 [31] ; Low for hsync pulse (32 cycles)
    set pins, 0 [31] ; Low for hsync pulse (64 cycles)
    set pins, 0 [31] ; Low for hsync pulse (96 cycles)

; BACKPORCH
backporch:
    set pins, 1 [31] ; High for back porch (32 cycles)
    set pins, 1 [12] ; High for back porch (45 cycles)
    irq 0 [1] ; Set IRQ to signal end of line (47 cycles)
.wrap
```



<https://vanhunteradams.com/Pico/VGA/VGA.html> 12

H sync generation



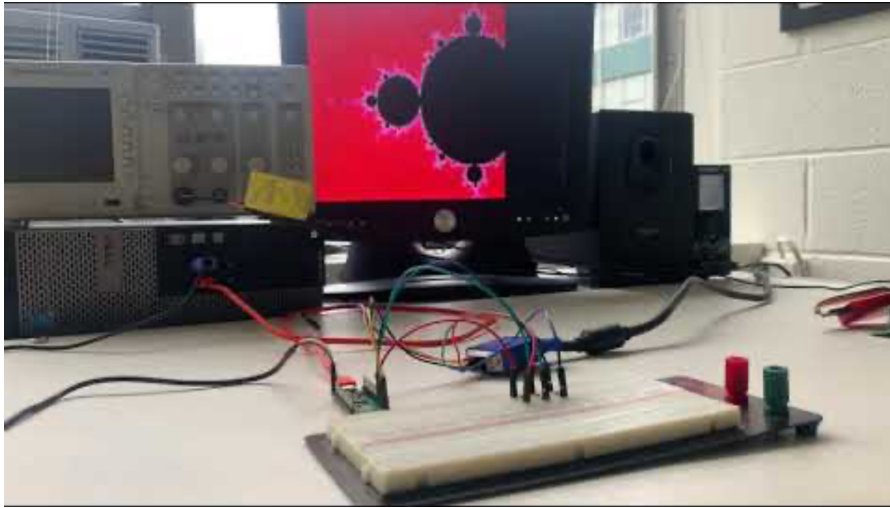
```
pull block ; Pull from FIFO to OSR (only happens once)
.wrap_target ; Program wraps to here

; ACTIVE + FRONTPORCH
mov x, osr ; Copy value from OSR to x scratch register
activeporch:
    jmp x-- activeporch ; Remain high in active mode and front
porch

; SYNC PULSE
pulse:
    set pins, 0 [31] ; Low for hsync pulse (32 cycles)
    set pins, 0 [31] ; Low for hsync pulse (64 cycles)
    set pins, 0 [31] ; Low for hsync pulse (96 cycles)

; BACKPORCH
backporch:
    set pins, 1 [31] ; High for back porch (32 cycles)
    set pins, 1 [12] ; High for back porch (45 cycles)
    irq 0 [1] ; Set IRQ to signal end of line (47 cycles)
.wrap
```

Generating a Mandelbrot set



DMA + PIO Example

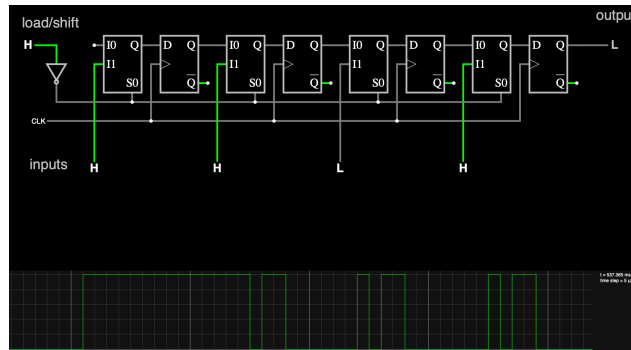


UNIVERSITY
OF ALBERTA

15

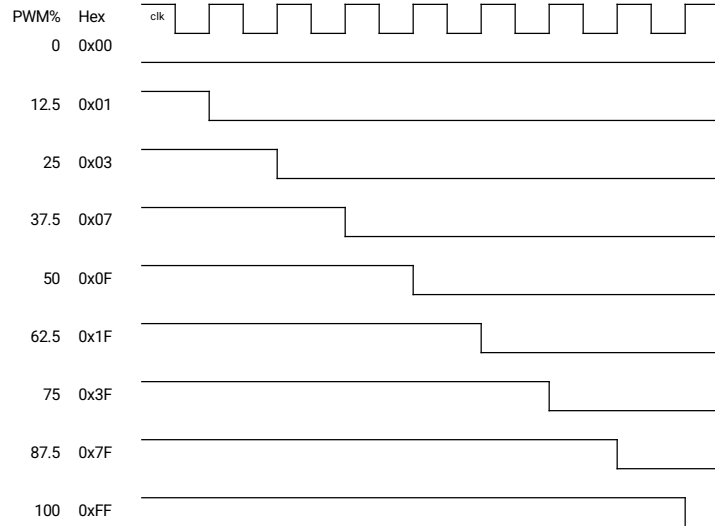
Use PIO to output PWM signal to LED

- The idea is to output bits on a GPIO pin at a constant rate so that the resulting signal is a form of pulse-width modulation (PWM)
- Using a parallel in, serial out (PISO) shift register, this is be easily accomplished



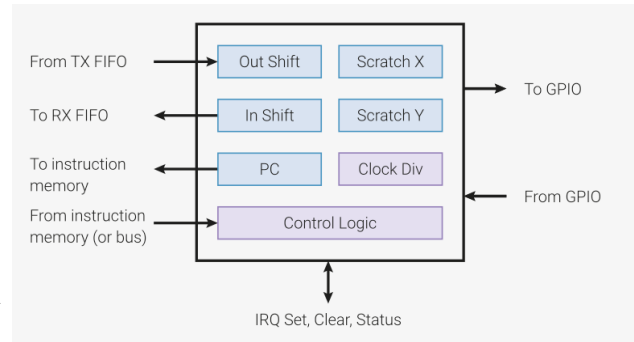
Use PIO to output PWM signal to LED

- Consider an 8-bit POSI shift register.
- Expressed in hex, the PWM widths are fractions of 8



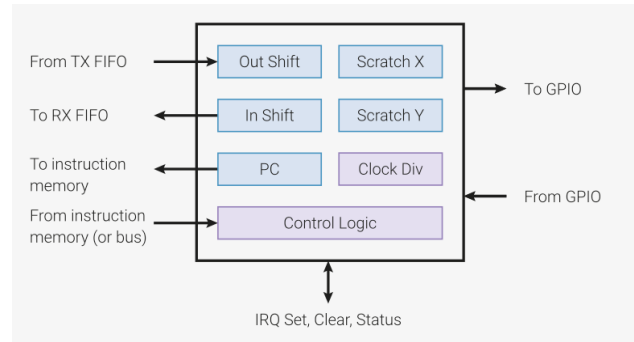
Use the RP2040 PIO Shift Register

- We can use the PIO output shift register (OSR)
- OSR connected to a GPIO
- OSR starts with 32-bit PWM pattern
- Bits are shifted out to the GPIO that is attached to an LED to turn it on a specific percentage of time
- Every 32 PIO clock cycles fill the TX FIFO, which is then copied into the SR when the register is empty



Use the RP2040 PIO Shift Register

- Set the PIO clock to be 10 system clock cycles
- Use 32 bits, so 32 PWM levels
- Outputting the 32 bits of one level will take
$$32 * 10 / f_{clk} = 320 / 125e6 = 2.56\mu s$$
- Going through all 32 levels will take
$$32 * 2.56 = 81.92\mu s$$



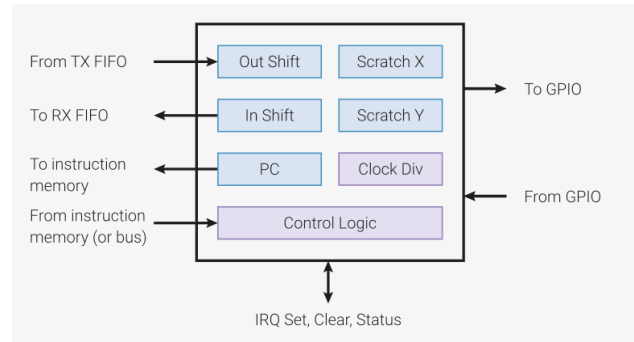
Use the RP2040 PIO Shift Register

- We want the 32 levels to take 2 seconds in total so the LED changing can be seen (it's not too fast)

- Thus, we need to repeat each level

$$2s / 81.92\mu s \approx 24414 \text{ times}$$

- Let's look at the program on Wokwi



PIO Program

```
;
; Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
;
; SPDX-License-Identifier: BSD-3-Clause
;

.program pio_serialiser

; Just serialise a stream of bits. Take 32 bits from each FIFO record. LSB-first.

.wrap_target
    out pins, 1
.wrap

% c-sdk {
static inline void pio_serialiser_program_init(PIO pio, uint sm, uint offset, uint data_pin, float clk_div) {
    pio_gpio_init(pio, data_pin);
    pio_sm_set_consecutive_pindirs(pio, sm, data_pin, 1, true);
    pio_sm_config c = pio_serialiser_program_get_default_config(offset);
    sm_config_set_out_pins(&c, data_pin, 1);
    sm_config_set_fifo_join(&c, PIO_FIFO_JOIN_TX);
    sm_config_set_clkdiv(&c, clk_div);
    sm_config_set_out_shift(&c, true, true, 32);
    pio_sm_init(pio, sm, offset, &c);
    pio_sm_set_enabled(pio, sm, true);
}
```

PIO Program

```
;
; Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
;
; SPDX-License-Identifier: BSD-3-Clause
;

.program pio_serialiser

; Just serialise a stream of bits. Take 32 bits from each FIFO record. LSB-first.

.wrap_target
    out pins, 1
.wrap

% c-sdk {
static inline void pio_serialiser_program_init(PIO pio, uint sm, uint offset, uint data_pin, float clk_div) {
    pio_gpio_init(pio, data_pin);
    pio_sm_set_consecutive_pindirs(pio, sm, data_pin, 1, true);
    pio_sm_config c = pio_serialiser_program_get_default_config(offset);
    sm_config_set_out_pins(&c, data_pin, 1);
    sm_config_set_fifo_join(&c, PIO_FIFO_JOIN_TX);
    sm_config_set_clkdiv(&c, clk_div);
    sm_config_set_out_shift(&c, true, true, 32);
    pio_sm_init(pio, sm, offset, &c);
    pio_sm_set_enabled(pio, sm, true);
}
```

OUT instruction shifts data from the OSR to other destinations, 1...32 bits at a time. In this case, 1 bit at a time

PIO Program compiled

```
// -----  
// This file is autogenerated by pioasm; do not edit!  
// -----  
  
#pragma once  
  
#if !PICO_NO_HARDWARE  
#include "hardware/pio.h"  
#endif  
  
// -----  
// pio_serialiser  
// -----  
  
#define pio_serialiser_wrap_target 0  
#define pio_serialiser_wrap 0  
#define pio_serialiser_pio_version 0  
  
static const uint16_t pio_serialiser_program_instructions[] = {  
    // .wrap_target  
    0x6001, // 0: out pins, 1  
    // .wrap  
};
```

A single instruction(!)

PIO Program compiled

```
#if !PICO_NO_HARDWARE
static const struct pio_program pio_serialiser_program = {
    .instructions = pio_serialiser_program_instructions,
    .length = 1,
    .origin = -1,
    .pio_version = pio_serialiser_pio_version,
#ifdef PICO_PIO_VERSION > 0
    .used_gpio_ranges = 0x0
#endif
};

static inline pio_sm_config pio_serialiser_program_get_default_config(uint offset) {
    pio_sm_config c = pio_get_default_sm_config();
    sm_config_set_wrap(&c, offset + pio_serialiser_wrap_target, offset + pio_serialiser_wrap);
    return c;
}

static inline void pio_serialiser_program_init(PIO pio, uint sm, uint offset, uint data_pin, float clk_div)
{
    pio_gpio_init(pio, data_pin);
    pio_sm_set_consecutive_pindirs(pio, sm, data_pin, 1, true);
    pio_sm_config c = pio_serialiser_program_get_default_config(offset);
    sm_config_set_out_pins(&c, data_pin, 1);
    sm_config_set_fifo_join(&c, PIO_FIFO_JOIN_TX);
    sm_config_set_clkdiv(&c, clk_div);
    sm_config_set_out_shift(&c, true, true, 32);
    pio_sm_init(pio, sm, offset, &c);
    pio_sm_set_enabled(pio, sm, true);
}
#endif
```

Called by main.c



24

main.c

```
#include <stdio.h>
#include "hardware/dma.h"
#include "hardware/irq.h"
#include "pio_seriliser.pio.h"

// PIO sends one bit per 10 system clock cycles. DMA sends the same 32-bit
// value 10 000 times before halting. This means we cycle through the 32 PWM
// levels roughly once per second.
#define PIO_SERIAL_CLKDIV 10.f
#define PWM_REPEAT_COUNT 24414
#define N_PWM_LEVELS 32

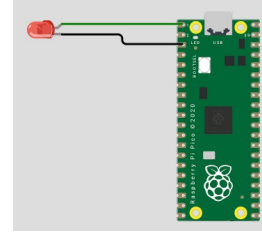
#define LED_PIN 0

int dma_chan;

void dma_handler() {
    static int pwm_level = 0;
    static uint32_t wavetable[N_PWM_LEVELS];
    static bool first_run = true;
    // Entry number 'i' has 'i' one bits and '(32 - i)' zero bits.
    if (first_run) {
        first_run = false;
        for (int i = 0; i < N_PWM_LEVELS; ++i)
            wavetable[i] = ~(~0u << i);
    }

    // Clear the interrupt request.
    dma_hw->ints0 = 1u << dma_chan;
    // Give the channel a new wave table entry to read from, and re-trigger it
    dma_channel_set_read_addr(dma_chan, &wavetable[pwm_level], true);

    pwm_level = (pwm_level + 1) % N_PWM_LEVELS;
}
```



main.c



```
int main() {
    // Set up a PIO state machine to serialise our bits
    uint offset = pio_add_program(pio0, &pio_serialiser_program);
    pio_serialiser_program_init(pio0, 0, offset, LED_PIN, PIO_SERIAL_CLKDIV);

    // Configure a channel to write the same word (32 bits) repeatedly to PIO0
    // SM0's TX FIFO, paced by the data request signal from that peripheral.
    dma_chan = dma_claim_unused_channel(true);
    dma_channel_config c = dma_channel_get_default_config(dma_chan);
    channel_config_set_transfer_data_size(&c, DMA_SIZE_32);
    channel_config_set_read_increment(&c, false);
    channel_config_set_dreq(&c, DREQ_PIO0_TX0);

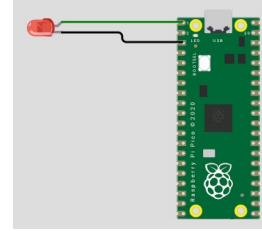
    dma_channel_configure(
        dma_chan,
        &c,
        &pio0_hw->txf[0], // Write address (only need to set this once)
        NULL,             // Don't provide a read address yet
        PWM_REPEAT_COUNT, // Write the same value many times, then halt and interrupt
        false             // Don't start yet
    );

    // Tell the DMA to raise IRQ line 0 when the channel finishes a block
    dma_channel_set_irq0_enabled(dma_chan, true);

    // Configure the processor to run dma_handler() when DMA IRQ 0 is asserted
    irq_set_exclusive_handler(DMA_IRQ_0, dma_handler);
    irq_set_enabled(DMA_IRQ_0, true);

    // Manually call the handler once, to trigger the first transfer
    dma_handler();

    // Everything else from this point is interrupt-driven. The processor has
    // time to sit and think about its early retirement -- maybe open a bakery?
    while (true)
        tight_loop_contents();
}
```



26

Wokwi time... ECE 315 PIO DMA Example

<https://wokwi.com/projects/449636130324734977>

Summary

- Showed how RP2040 PIO can be used to generate complex hardware waveforms, barely using the CPU
 - In the example, the CPU was used mostly to compute the Mandelbrot set
- After review of VGA signals, showed how one PIO state machine (SM) is used to generate H Sync, one generates V Sync, the the third SM clocks out the RGB signals
- A second example using only DMA and PIO, no CPU, to cycle an LED through 32 PWM levels
 - Used the clock divider to slow the PIO and repeated DMA transfers to make the PWM ramping “human speed”

Next Lecture

- Using DMA to Capture Sensor Data

Questions

